

Astrophysical Recipes (Second Edition)

The art of AMUSE

Simon Portegies Zwart, Steve McMillan and Steven Rieder

Chapter 10

Expanding and Extending AMUSE

No limit may be set to art,
neither is there any craftsman that is fully master of his craft.
—The Instruction of Ptahhotep

Congratulations on reaching the final chapter of this book! In the last several chapters, we have demonstrated how AMUSE can be used to run individual codes and, more importantly, how to couple codes to simulate more complex systems. In this final chapter, we present several examples of advanced strategies for coupling multiple codes trading performance for accuracy, and vice versa. In addition, we explore two tightly coupled physical problems in more depth. In the first example, we explore accretion onto a supermassive black hole from a population of massive stars in orbit around it. In the second, we study the effect of a nearby supernova on a young planetary system with a circumstellar disk. The problems in this chapter require considerable computer resources for high-resolution results, but the examples can be run at lower but acceptable resolution with some patience on a good workstation.

10.1 Intricate Multiscale Coupling Strategies

The stars in a cluster are often found in pairs, triples, or even higher-order hierarchies. Democratic hierarchies are notoriously unstable and fall apart into smaller components. In some rare cases, higher-order hierarchies can be stable for a longer time, but eventually most of them dissolve. The fundamental building blocks of a star cluster are ultimately single stars and binaries.

So far, we have only discussed binaries in our discussion on stellar evolution (Chapter 5, Sections 8.7.1 and 8.7.2, and Section 9.4.5). In this chapter, we will discuss some of the numerical complications with binaries. The terms binaries and

higher-order hierarchies should be read here in their broadest context because we will use these terms to address planetary systems, stars around a supermassive black hole, and ordinary stellar multiples.

Within the AMUSE framework, we have two fundamental quite different methods to deal with hierarchical systems as part of a large star cluster. We discuss them separately in Sections 10.1.1 and 10.1.2.

In each case, we make slightly different assumptions which help us to resolve the dynamics of the multiple systems in a larger stellar environment. As a rule of thumb, we could argue that the `encounters` module (see Section 10.1.1) is meant for simulating star clusters with lots of binaries, whereas the `Nemesis` module (see Section 10.1.2) is used for integrating planetary systems including moons and asteroids inside a star cluster.

Although we present here two methods in which hierarchical systems can be resolved in a large environment, other strategies are possible. One such possibility, effectively employed in several studies, is the use of Kepler's second law (Kepler 1609) to resolve close binaries in a star cluster. The interaction with other stars can then be incorporated as perturbations applied externally onto the binary system. With the various bridging strategies and the information from this chapter, it is quite easy to construct a hybrid integrator in which Kepler solutions replace some of the orbital integrations.

In the final two sections, we discuss various modules developed within the framework. These include the `Encounters` (Section 10.1.1), `Nemesis` (Section 10.1.2), `Venice` (Section 10.1.3), `Torch` (Section 10.2.1), and `Ekster` (Section 10.2.2), and `Fresco` (Section 10.5.1) modules.

10.1.1 Encounters: Multiples in Stellar Clusters

The `encounters` module is based on the principle that encounters between a number (>2) of stars are short-lived compared to the global evolution timescale of the cluster. And that, if such an encounter is long-lived, it can be separated into a number of shorter encounters. The underlying assumption of the `encounters` module then is that any encounter is instantaneous with respect to the cluster's dynamical evolution.

The crossing time for a typical stellar cluster is on the order of a few million years (see Section 4.1.2), whereas a 100 au encounter between two solar-mass stars lasts a few thousand years. A democratic resonance encounter does not necessarily last much longer, unless it is associated with wide excursions to large distance in a hierarchical resonance. A star in such an excursion-orbit may then be perturbed again by another star in the cluster, and we argue that such encounters can be separated into individual encounters.

Binaries and multiples can form dynamically in the cluster; but in many cases, we want to start with a population of primordial binaries. In Code example 10.1, we demonstrate how binary stars are set up in the `multiples` module. It encompasses three particle sets, one for the single stars (not in binaries), one for the binary centers of mass, and one for the stars in binaries. Note that the latter are replicated (as

```

137     for bi in binaries:
138         mp = bi.mass
139         ms = np.random.uniform(mmin.value_in(units.MSun),
140                                 mp.value_in(units.MSun)) | units.MSun
141
142         nb = new_binary_orbit(mp, ms, bi.semi_major_axis, bi.eccentricity)
143         nb.position += bi.position
144         nb.velocity += bi.velocity
145         nb = singles_in_binaries.add_particles(nb)
146         nb.radius = ZAMS_radius(bi.mass)
147         nb.type = "planet"
148         nb.name = "planet"
149         binary_particle = bi.copy()
150         binary_particle.child1 = nb[0]
151         binary_particle.child2 = nb[1]
152         resulting_binaries.add_particle(binary_particle)
153
154     single_stars = center_of_masses-binaries
155     return single_stars, resulting_binaries, singles_in_binaries

```

Code example 10.1: Setting up binaries for the multiples module. The full code example is in `amuse.examples.multiples_minimal`.

pointers) in the binary center of mass particle set as `child1` and `child2`. The particle set provided (called `binaries`) contains three attributes which are relevant for setting up the binaries and for the operation of the multiples module, these are the `semi_major_axis`, `eccentricity`, and the `radius`. The first two are used to set up the binary, and could equally well be provided externally. The `radius`, however, is an important parameter that tells the `multiples` module within what distance a binary should be considered isolated. Here, we adopt three times the apocenter distance, but this parameter is problem-dependent and probably should evolve at runtime.

Once the stars and binaries are generated, we can set up the code structure. When two or more stars in an evolving star cluster approach each other within this radius, we stop the global cluster integration. A new N -body code is started within which the encountering stars are integrated until the system dissolves again. The resolved system is subsequently put back in the global N -body integrator and the interim integrator is deleted.

In Code example 10.2, we present the fundamental setup for the `multiples` module. This starts with generating the converter and starting the N -body code. We subsequently start the `encounters` handling module and the `multiples` module. The `encounters` module has two important arguments: the code used for resolving two-body encounters and the code for resolving multibody ($\gtrsim 3$) encounters. For the former, we use the standard Kepler solver, and for the latter, we use the `SmallN` solver in `AMUSE`.

The `multiples` module also has two handling codes: one is the gravity code for integrating the global system, and the other is the encounters module, which we have just defined. For the gravity module, we use `ph4` in this example, but it could be any N -body code.

code font

```

201 converter=nbody_system.nbody_to_si(stars.mass.sum(), t_end)
202 gravity = Ph4(converter)
203 gravity.parameters.force_sync = True
204
205 encounter_code = encounters.HandleEncounter(
206     kepler_code=new_kepler(converter),
207     resolve_collision_code=new_smalln(converter),
208     interaction_over_code=None,
209     G=constants.G
210 )
211 multiples_code = encounters.Multiples(
212     gravity_code=gravity,
213     handle_encounter_code=encounter_code,
214     G=constants.G
215 )

```

Code example 10.2: Setting up the multiples module.

```

219 multiples_code.particles.add_particles(stars.copy())
220 multiples_code.singles_in_binaries.add_particles(planets_in_JuMBOs)
221 multiples_code.binaries.add_particles(JuMBOs)
222 multiples_code.commit_particles()
223
224 stopping_condition \
225     = multiples_code.stopping_conditions.binaries_change_detection
226 stopping_condition.enable()

```

Code example 10.3: Providing the particles to the multiples module.

In Code example 10.3, we demonstrate how to add the various particles to the multiples module. Here, three types of particles are included: the stars, the planets_in_JuMBOs, and the JuMBOs. The stars are all the gravitational objects handed to the code. The planets_in_JuMBOs include the particles which are the member of a binary system, and the JuMBOs, in this example, represent the centers of mass of the binaries. The term JuMBO may be a bit arcane here, but is related to the recent discovery of Jupiter-mass binary objects in the direction of Orion's Trapezium cluster (Pearson & McCaughrean 2023). On the other hand the detection of JuMBOs were not confirmed, and the original paper was never published. The scripts to stimulate the dynamical evolution of these hypothetical JuMBOs can be used to demonstrate why these objects have no chance to survive in the violent dynamical environment of a young stellar cluster.

One advantage of the multiples module is the strict separation of binaries (and multiples) from the single stars. This allows us to invoke binary and triple evolution by treating those substructures with separate methods compared to single stellar evolution. An example of this is presented in the kira code in Code example 10.4 and Section 8.7.2.

Although the encounters module is quite powerful and speeds up N -body codes with a large number of binaries or multiples, it is best suited to the rather specific problem of relatively separated multiple encounters in a stellar cluster. The assumption that the rest of the cluster is not evolving while the subsystem experiences an interaction may break down when an encounter lasts for a substantial

```

1  resulting_binaries = Particles()
2  singles_in_binaries = Particles()
3  binaries = center_of_masses.random_sample(Nbin)
4  mmin = center_of_masses.mass.min()
5  for bi in binaries:
6      mp = bi.mass
7      ms = (
8          np.random.uniform(mmin.value_in(units.MSun), mp.value_in(units.MSun))
9          | units.MSun
10     )
11     a = random_semimajor_axis_PPE(mp, ms)
12     e = np.sqrt(np.random.random())
13
14     nb = new_binary_orbit(mp, ms, a, e)
15     nb.position += bi.position
16     nb.velocity += bi.velocity
17     nb = singles_in_binaries.add_particles(nb)
18     nb.radius = 0.01 * a

```

Code example 10.4: Example of how to use the encounter module.

amount of time, and when many encounters occur simultaneously. An example would be a star cluster in which the majority of stars are orbited by planets, maybe including some binary stars. To address such a complex multiscale but rather narrow range of physical processes, we developed the more general *Nemesis* module, which we discuss in the following section.

10.1.2 *Nemesis*: Planetary Systems in Stellar Clusters

What the *multiples* module does for binary stars or more hierarchical multiples, *Nemesis* does for systems with one or more dominating body and several minor bodies, such as a planetary system. The main advantage of *Nemesis* over *Encounters* is the consistent time integration. *Encounters* assumes interactions between multiple subsystems to be instantaneous, whereas in *Nemesis* time progresses gradually and continuously, although objects can switch from child to parent system, and vice versa, on a local crossing timescale.

The realization that many stars are potentially born with multiple planets (and smaller objects) begs for a numerical treatment of such hierarchical systems. Stellar distributions around supermassive black holes are quite similar to planetary systems, except that the distributions of orbital eccentricity and inclinations of stars around a supermassive black hole tend to have a wider range compared to planetary systems.

The relatively short orbital periods of planetary systems with respect to the crossing time of a star cluster gives rise to a number of interesting numerical challenges. One of these is the time to solution, which, due to the short time steps for integrating planets, calls for multi-time-step algorithms. A disadvantage of these algorithms, however, is their lack of symplecticity.

In *Nemesis*, each planetary system is integrated with its own individual integrator. When several stars or planetary systems approach each other within a prespecified limit, both systems are merged and further integration is realized by a single integrator. If a multibody system dissolves, single objects are picked up by the global *N*-body integrator, whereas the multibody subsystems continue to be

integrated with their individual solvers. Integrators in this method are dynamically created and deleted at runtime. *Nemesis* is fast, efficient, parallel, and effective in addressing multiplanetary systems in a star cluster (van Elteren et al. 2019). In addition, it is possible for each subsystem to be addressed with its own private integrator. This flexibility makes *Nemesis* quite unique, and applicable to a wide range of problems. One could even address one system with a hydrodisk, while around another star the disk has already coagulated into a planetary system. In principle, a child system could itself be a version of *venice* or *multiples*, or even a complete magneto radiative hydrodynamical simulation. Here, however, we focus the dynamical aspects of *Nemesis*.

In *Nemesis*, we integrate planetary systems and stars together, but not entirely self-consistently. Planets or planetesimals on wide orbits may move between their host child (the planetary system) and the parent system (star cluster). As a consequence, that particular particle will be integrated for parts of its orbit with different integrators. An example would be an Oort-cloud object orbiting a star which is a cluster member. The planetesimal then will be integrated in the child system when near pericenter, say within 1000au, but beyond a certain point it will be considered part of the parent system. The logic associated with this child-to-parent transition is, despite being parallelized, expensive in the current implementation of *AMUSE*. In addition, the child system will lose its symplectic qualities by regularly adding and removing particles. The errors associated with these changes are small, but noticeable for long timescale integrations.

Child systems are composed of small numbers of stars, planets, and other minor bodies. These stars and planets have a relative interaction time step smaller than some fraction η of the free-fall time. The parent system (including the collective of child system centers of mass) is typically integrated using a fourth-order Hermite code, or maybe a treecode. Each child system is integrated using a high-order N -body code for small- N systems, such as *rebound* (Rein & Liu 2012), *Mercury* (Chambers & Migliorini 1997), or *Huayno* (Pelupey et al. 2012).

In calculating the force on the center of mass of a subsystem, we take the substructure of the other subsystems self-consistently into account. This procedure ignores the gravity of the child system’s minor bodies on the internal evolution of each of the other child systems’ minor bodies, and vice versa. While integrating the Alpha Centauri system, we would then, for example, ignore the gravitational effect of planetesimals in the solar system. The errors that result from this procedure can be estimated, and turn out to be negligible, as expected.

Nemesis is implemented in a number of bridge schemes (see Chapter 9). These bridges integrate the equations of motion of the individual components (collective and subsystems) self-consistently via a second-order kick-drift-kick (KDK) method. One advantage is that the entire method is independent of the integrators used for either the collective or of the subsystems. Individual bridges are needed between the cluster and the center-of-mass subsystems, and between all subsystems and the cluster. These bridges connect the split parts by employing a second-order KDK method. Kicks are calculated from the gravitational field of the kicking cluster, and drifts are the evolution of the models in the separate N -body solvers (Portegies Zwart et al. 2020).

The bridge between the cluster and each subsystem needs to take into account that the subsystem is represented in the cluster by a center-of-mass particle. Two adaptations to a vanilla KDK method are needed for this. First, to avoid an extra self-gravity force, the kick has to be calculated with all particles except the center-of-mass particle of the subsystem in question. Next, to avoid doubly calculating the gravitational field of the cluster on the ensemble of the particles, once during the evolution of the particles in the direct N -body code and once with this kick, the kick on the central particle from the cluster must be subtracted from all kicks on the particles in the subsystem.

The bridge between the subsystems and the cluster needs similar adaptations. Now everything is reversed; the particles in each subsystem kick the center-of-mass particles in the cluster, except the one representing the subsystem. The kick of the center-of-mass particle of each subsystem is subtracted from the sum of the kicks of each particle in the subsystem. Figure 10.1 presents a schema of the method. Of all the possible interactions between the particles, we ignore the forces between the particles of one subsystem and another.

Although we have accounted for the interactions between the cluster and the subsystems, we do not yet have a working method because each kick on a subsystem will result in a velocity component of the center of mass of that subsystem. The same is true of each drift phase: the N -body solvers should conserve the center-of-mass position and velocity but small errors will cumulatively result in an increasing position offset and velocity component. After the first kick and for each drift of a KDK step, we need to absorb the translation of the center-of-mass velocity and position into the center-of-mass particle in the cluster. This is not necessary for the

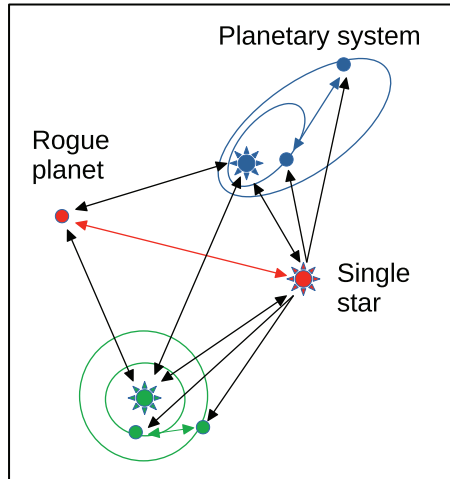


Figure 10.1. Strategy to reduce computing cost without loss of accuracy in Nemesis. The planets in one system (blue) interact with the center of mass of another system (green) but not with its planets. Single stars and free-floating planets are marked in red.

second kick because the kick is independent of the velocity component and this component will be resolved in the first kick of the next step.

Dividing the cluster in this way introduces a new problem: the gravitational evolution of each subsystem can disintegrate or subdivide the subsystem. The subsystem would grow in size and no longer evolve on a separate space scale as the cluster. A clustering algorithm is used to determine for each subsystem if it needs to be restructured in one or more new subsystems. This algorithm is applied after each KDK step.

Similar to subsystem disintegration, subsystems can merge to form a new subsystem. We detect these mergers by defining a radius on each center-of-mass particle and interrupting the drift phase of the center-of-mass system whenever the distance between two of these particles is smaller than the sum of the individual radii. Before merging the subsystems, we need to bring the state of each system to the merge time by applying a kick and drift on each subsystem. The kick will be subtracted if the time is before half the step time and added otherwise. Next, each system is added to the new subsystem and the new center-of-mass particle is added to the cluster. The drift of the cluster can then continue as normal.

10.1.3 Venice: A Generalized Multiscale Bridge

Bridge is powerful, but limited by the rigidity of its implementation. These limitations include the order of the integration and the constant time step. Both can be relaxed. Here, we show how these limitations can be mitigated by using a generalization we call *Venice* (Wilhelm & Portegies Zwart 2024).

Venice is an operator-splitting algorithm designed to address the challenges of integrating numerical models across multiple timescales in multiphysics simulations. As in *bridge*, *Venice* is a multiphysics coupling strategy for combining a diverse collection of physical processes operating on vastly different timescales.

The traditional *bridge* approach becomes computationally expensive and complex due to the need to synchronize all processes on the shortest common time step. In *Venice*, we overcome these limitations by enabling adaptive coupling of independent components on their individually relevant timescales, thereby improving both performance and maintainability.

In *Venice*, we connect all available codes in a matrix of bridges. Each bridge coupling is associated with an interaction timescale, resulting in a matrix of timescales. The coupling is second-order, as in classic *bridge*, but in principle this is easily generalized. Dynamically changing the order of individual bridges, however, is currently not supported within *Venice*. The coupling strategy, however, is considerably more flexible than in the traditional *bridge*. In searching for a name for an elegant method representing a matrix of bridges, we settled on *La Serenissima*: the Italian city of canals.

The adaptive time-stepping allows all components to operate on an individual natural timescale. As in Equation (9.3), we split the general operator and solve each part separately. After solving the individual parts, we combine them into a unique, self-consistent solution. The mathematics is a bit daunting, for which we refer to

Wilhelm & Portegies Zwart (2024). With Venice in mind, one could perceive bridge as a simpler bidirectional strategy.

The main advantages of Venice include its temporal flexibility and the modular design, making it easier to maintain and extend simulation frameworks and optimize multiscale and multiphysics solvers. The algorithm supports an arbitrary number of operators and coupling strategies, making it suitable for a wide range of multiphysics problems.

To make Venice work properly in AMUSE, we use channels to copy the data from one code to another. These are defined directly between codes and allow any subset of data to be copied from one code to the other. They must be explicitly called, which is done between operations. In a more relaxed approach, the copying could be automatically realized after the call to the specific code, but this would result in an end state where the data of different codes is not synchronized, in which case it is nontrivial to determine which code contains the most recent state. The explicit approach, where we copy all data from a code after its evolution step, introduces some unnecessary copy operations, but is less prone to mistakes. Of course, this can be optimized, in particular if one takes the multiscale modeling paradigms (Borgdorff et al. 2011) properly into account.

As an example of how Venice operates, we dynamically evolve a stellar cluster with a few very massive stars. The fundamental setup can be written as follows

```
from venice import Venice
def make_gravity_stellar (converter, timestep, verbose=False):
    stellar = Seba()
    gravity = Ph4(converter)
    gravity.parameters.force_sync = True

    system = Venice()
    system.verbose = verbose
    system.add_code(stellar)
    system.add_code(gravity)

    system.add_channel(0, 1, from_attributes=["mass"], to_attributes=["mass"])
    system.timestep_matrix[0,1] = timestep
    return stellar, gravity, system
```

First, we initialize both codes for stellar evolution and dynamics. Note that we added the line

```
gravity.parameters.force_sync = True
```

to guarantee that the N -body code properly synchronizes to the requested time (see also Section 8.4.4). Some N -body codes do not naturally synchronize to precisely the time requested because it introduces extra numerical errors, slows down the code, and is often not necessary. For Venice, however, we do require the codes to be synchronized at the end of a step.

The actual Venice system is initiated in much the same way we initialize a bridge in the lines

```
system = Venice()
system.add_code(stellar)
system.add_code(gravity)
```

Here, the script is started, and the (in this case) two codes are added. Running the code is not very different than a usual code bridge.

```
timesteps = [3., 1., 0.3, 0.1, 0.03, 0.01, 0.003, 0.001] | units.kyr
stellar_l, gravity_l, system_l = make_gravity_stellar(converter, timesteps[i])

stellar_l.particles.add_particles(cluster_l_copy)
gravity_l.particles.add_particles(cluster_l_copy)
channel_l = gravity_l.particles.new_channel_to(cluster_l_copy)

stellar_i, gravity_i, system_i = make_gravity_stellar(
    converter,
    timesteps[i]
)
system_i.interlaced_drift = True

stellar_i.particles.add_particles(cluster_i_copy)
gravity_i.particles.add_particles(cluster_i_copy)
channel_i = gravity_i.particles.new_channel_to(cluster_i_copy)

system_l.evolve_model(end_time)
channel_l.copy()
system_l.stop()
```

Subsequently, running the code is not much different than any other code in AMUSE.

With Venice, we can couple an arbitrary number of codes and run them on their own preferred timescales. If you encounter an astrophysical problem in which two codes are coupled, Venice is probably overkill, but you should consider using this strategy once more than two codes require coupling. In Figure 10.2, we present an example of the working of Venice, showing the evolution of the stellar time step and type as a function of time.

10.2 Advanced Coupling Projects

10.2.1 Torch: Forming Star Clusters from Molecular Clouds

One of the objectives of the MODEST community is to model the formation and evolution of a star cluster from the moment a giant molecular cloud starts to collapse. This has led to the Torch project,¹ in which AMUSE is used as coupling framework to model star cluster formation by integrating multiple simulation codes.

Installing Torch is nontrivial. After AMUSE is properly installed, the MHD code FLASH needs to be installed, after which the Torch package can be added. You can download an academic version of FLASH from <http://flash.uchicago.edu/site/flashcode>.

If not all packages are installed in a conda environment, you may have to set some environment variables in your shell

¹ See <https://torch-sf.bitbucket.io/quickstart.pdf>

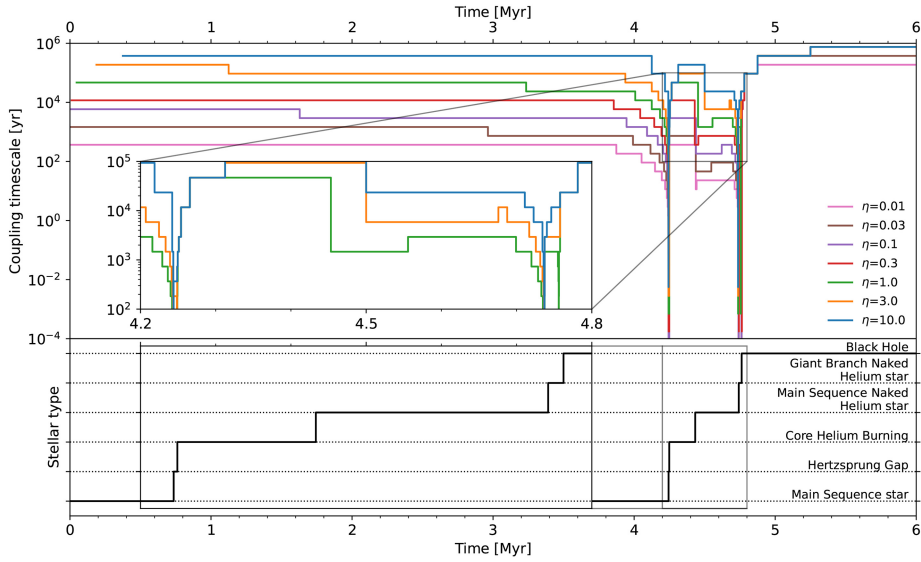


Figure 10.2. Illustration of the dynamic coupling timescale between gravitational dynamics and stellar evolution (top panel). The calculation is performed for a range of coupling timescales η from 0.01 to 10. The bottom panel illustrates the type of the most massive star in the cluster (of $50 M_{\odot}$) star in time. We illustrate how different coupling timescales lead to slightly different moments the most massive star in the cluster is updated (see Wilhelm & Portegies Zwart 2024). `amuse.examples.venice_minimal`. It should take about 10 minutes to run on a laptop.

```
export AMUSE_DIR=/path/to/amuse
export FLASH_DIR=/path/to/FLASH4.5
export TORCH_DIR=/path/to/torch
export PYTHONPATH=$PYTHONPATH:$TORCH_DIR/src
```

Then, run the installation run script:

```
./install.sh
```

You will be prompted to confirm your environment variables, and continue with the installation. The installer will then automatically copy interface code from `TORCH_DIR/src` to their correct locations in your AMUSE and FLASH repositories. It will also ensure that the FLASH4.5-a patch for HDF5 >1.10.x compatibility is applied in your FLASH repository. The main script is `torch_user.py`, but we encourage you to make a copy in some local directory and change it to your liking.

In Figure 10.3, we show the analytic energy injection of a pressure-dominated snowplow phase of a supernova explosion in a gaseous medium using the Torch simulation package. The various vertical lines give the Sedov–Taylor (t_{ST}), thermal (t_{TR}), and snowplow (t_{PDS}) transition times. Figure 10.3 demonstrates how in Torch these high-energy phenomena are satisfactorily represented.

The Torch module uses three codes coupled using the AMUSE framework. These codes are the MHD-radiative-transfer hydrodynamics code FLASH (Fryxell et al.

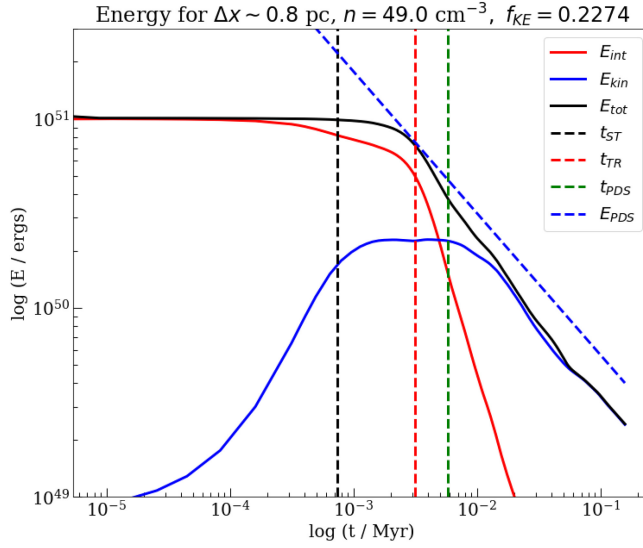


Figure 10.3. The Sedov–Taylor (t_{ST}), thermal (t_{TR}), and snowplow (t_{PDS}) transition times (Draine 2011) are shown as vertical dashed lines, and the analytic form of the energy during the pressure-dominated snowplow phase is shown as a blue dashed line above the energy during this phase. Reproduced from Wall et al. (2020). The Author(s). CC BY 4.0.

2000), the direct N -body code PeTar (Wang et al. 2020), and the binary evolution code SeBa (Portegies Zwart & Verbunt 1996). Although each of these codes could in principle be replaced by any other in AMUSE, the Torch script has become quite intricate, and by now it may not be trivial to replace, e.g., FLASH for Ramses (Teyssier 2002). On the other hand, it is somewhat easier to replace PeTar for the multiples module, and SeBa for Mesa, for example.

The performance and accuracy of PeTar are tuned using the parameter `r_out` which is a boundary radius between the tree- and direct N -body gravity. The default is tuned for simulating star clusters, but by all means test this and modify it for your particular use case. Torch can also use VorAMR (Lewis et al. 2023), a utility developed within the Torch framework that allows for the conversion of output data from AREPO to be converted to an AMR grid. An example is provided in Figure 2.1 of Section 2.2.2.

A Torch calculation starts with a giant molecular cloud that collapses under its own gravity. Star formation is mediated through sink particles. We use AMUSE’s sink-particles module to handling the sinks (see Section 6.3.2). Stars are spawned from these sinks according to a predetermined mass function. Stellar feedback is facilitated by the transfer of stellar parameters (luminosity, mass-loss rates) from SeBa to FLASH, where these parameters are used to calculate the effects of radiation, winds, and supernovae on the surrounding gas.

Recently, we added anisotropic winds, or jets to simulate the formation of low-mass clusters (Appel et al. 2025). Protostellar jets and outflows are critical phenomena associated with star formation, launched by the interaction of magnetic fields and accretion disks. In Figure 10.4, we present a test result of a simulation using jet outflows from a young stellar object in Torch.

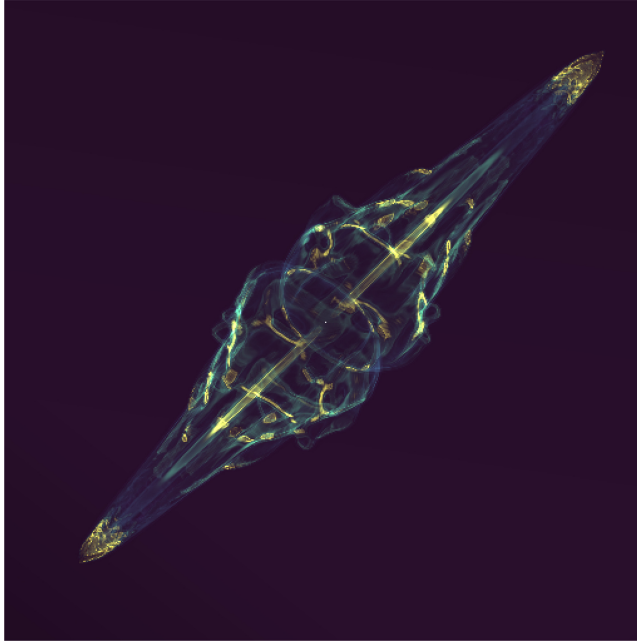


Figure 10.4. A volume rendering of a protostellar jet produced by a single star particle in a uniform density box. This volume rendering was generated from a simulation run using the jets branch of the Torch code. Note that although the injection weighting is smooth, as described in Appel et al. (2025), hydrodynamical instabilities develop as the injected material propagates into the denser background material.

10.2.2 Ekster: Forming Clusters from Gas to Star

The Ekster² module in AMUSE is somewhat similar to the Torch endeavor (Rieder 2019; Rieder & Liow 2021). Ekster has a similar philosophy as Torch, except that it uses an SPH method for the hydrodynamics. Ekster was developed with the SPH code Phantom (Price et al. 2018) as its primary hydrodynamics code, although following the AMUSE philosophy it is possible to use any other SPH code, provided it supports the high-density stopping condition, see Section A.5.2. Like Torch, Ekster combines hydrodynamics with gravity and stellar evolution, by default using the same codes as in Torch (PeTar and SeBa).

code
font

The application domain for Ekster is quite similar to that for Torch, as was illustrated in Sills et al. (2023). Ekster was developed with relatively low mass resolution in mind ($\sim 10^5 M_\odot$ per gas particle), allowing it to run on modest-sized machines (anything from a laptop to a small cluster). When gas reaches a sufficiently high density, star formation starts through the formation of sink particles, each of which acts as a star-forming region. The mass in the sinks is converted into stars, which are spatially distributed within the sink's accretion radius using a Gaussian distribution. The mass of the stars is sampled from a predetermined mass function.

² Ekster is the Dutch name for the European magpie. Also, it is a homophone for Exeter, where the module was primarily developed. Ekster is available from: <https://github.com/amusecode/amuse-ekster>.

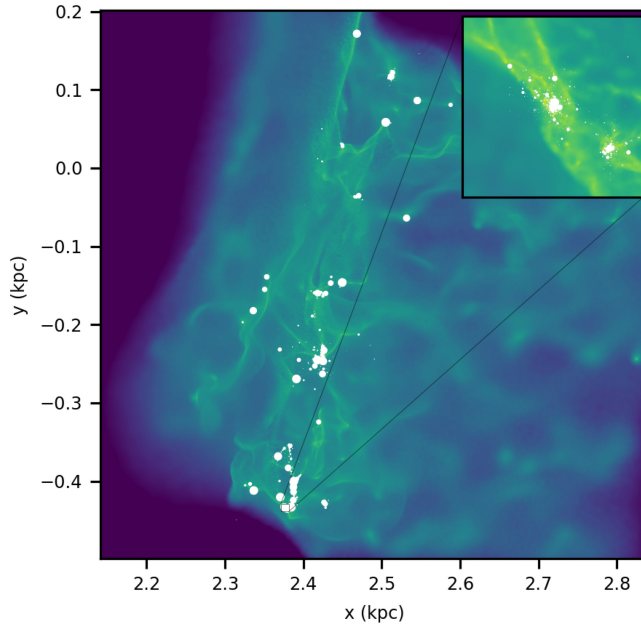


Figure 10.5. One of the regions CRC0060 at an age of 3 Myr (Rieder et al. 2022). The inset shows a 10 parsec zoom of the largest star cluster. Symbol sizes are proportional to the stellar radius. Adapted with permission from Rieder et al. (2022). © 2022 The Authors. Published by Oxford University Press on behalf of the Royal Astronomical Society.

After star formation, the sink’s accretion radius shrinks so that its “density” remains constant. This prevents the sink moving towards and accreting low-density gas regions.

As an alternative for simulations with a slightly higher mass resolution, *Ekster* also has a mode where multiple sinks can be grouped according to their positions, velocities, and ages. These groups can then “borrow” mass from each other for star formation. This allows massive stars to form even when individual sinks are unable to accrete enough mass for such stars (Liow et al. 2022).

In Figure 10.5, we show a small section of a simulated galaxy (Dobbs & Pringle 2013) for which the simulation was continued using *Ekster* in *AMUSE* (Rieder et al. 2022). It shows one of the active star-forming regions in the simulated galaxy at an age of 3 Myr after the collapse of the giant molecular cloud.

10.3 Supernova Impact on the Early Solar System

Meteoritic evidence suggests that short-lived radionuclides—specifically ^{26}Al and ^{60}Fe , with half lives of 0.717 and 1.49 Myr, respectively (Schiller et al. 2015)—were incorporated into grains in the primitive solar nebula about 2 Myr after those grains formed (Luu et al. 2015; Scott 2007). As discussed in Section 8.6.1, the origin of these unstable elements is uncertain, but one hypothesis is that a nearby supernova might have enriched the solar circumstellar disk with these radionuclides shortly

after the Sun formed. The process would have required high temperatures (1500–2000 K) to melt the grain material. In principle, the same supernova could have produced the ^{26}Al and ^{60}Fe radionuclides, delivered them to the disk, and heated the disk enough to melt the hardest grains. For this to work, the supernova must have exploded within about 0.5 pc and within the first 1.5 Myr of the budding solar system. The proximity is necessary to guarantee a sufficient amount of short-lived radionuclides, and the timescale stems from the half-life of the volatile elements.

It might seem extraordinary to posit a type II supernova so close to the Sun just 2 Myr after it formed. However, considering that the Sun was probably born in a star cluster, the probability of a such an event is actually quite high. Looney et al. (2006) argue that a $23 M_{\odot}$ star could deliver a sufficient amount of short-lived radionuclides when it explodes at an age of about 8.4 Myr. The time difference can plausibly be explained by an age spread in the star formation process, in which the Sun formed somewhat later than the most massive stars (Stahler & Palla 2005).

The possibility of a nearby supernova was first suggested by Chevalier (2000). Subsequent hydrodynamical calculations have been performed to study the amount of supernova remnant material that could be accreted by the protoplanetary disk (Ouellette et al. 2007). These simulations suggest that the supernova might not be able to deposit sufficient mass on the disk to match the observations, but made some simplifying assumptions and were not conclusive. One of those assumptions was homogeneous density throughout the slab of supernova material hitting the disk. Here, we take the opportunity to revisit the problem using AMUSE, combining gravitational dynamics, hydrodynamics, and radiative transfer codes to explore this scenario. Unlike previous studies, we investigate both the initial irradiation of the disk by the supernova flash and the direct effects of the subsequent blast wave, both of which could have contributed to the heating of the disk. Most of the ingredients of the scripts described below have already been discussed in previous chapters.

We start by generating a protoplanetary system with a star, some gas-giant planets, a gas disk, and debris. This system is evolved using a bridged hydrodynamics–gravity solver (see Section 9.3.2) in which we integrate the debris disk as test particles in the combined potentials of the planets and the gas disk. We ignite a supernova at a distance of $d_{\text{sn}} = 0.1$ pc and assume that the normal to the disk is inclined at an angle of 60° to the direction to the supernova, so the radiation and the blast wave impinge on the disk almost edge-on. A head-on blast wave would be less effective in tilting the disk, and would damage the disk more severely.

The supernova explosion is modeled in two stages. First, a radiative transfer code is started to irradiate the protoplanetary disk roughly $d_{\text{sn}}/c \sim 100$ days after the explosion. Then, approximately 10 years later, the blast wave arrives at a speed of $\sim 10,000 \text{ km s}^{-1}$. At the adopted small distance, the supernova shell is still in its free-expansion phase and the hot gas has not been significantly slowed by the interstellar medium. The short distance, and therefore short travel, time guarantees that the short-lived radionuclides are still abundant in the blast wave. We study the effect of the supernova irradiation and the subsequent blast wave on the disk, and determine if the disk survives and if a sufficient amount of short-lived radionuclides can be deposited.

10.3.1 Initial Conditions and Model Parameters

We start the simulation by generating a young solar system with four giant planets using the method discussed in Section 4.4.1. The underlying assumption is that the rocky planets have not yet emerged, so instead we incorporate a debris disk. The solar system was generated at an arbitrary Julian date of 2474649.5 days³ After initialization, we recenter the system to place the center of mass at the origin.

The gas disk around the star is generated using $n_{\text{disk}} = 10000$ equal-mass SPH particles between $r_{\text{min}} = 1$ au and $r_{\text{max}} = 100$ au from the Sun, with total mass of $0.01 M_{\odot}$. We adopt a flat density profile with Safronov–Toomre Q-parameter $q_{\text{out}} = 25$ (see Section 6.4.1). You might wish to experiment with higher resolution, but note that the calculation is quite expensive in terms of computer time, and parallelization is limited to the OpenMP implementation of the adopted hydrodynamics solver (see below).

```
converter = nbody_system.nbody_to_si(mass_star, radius_min)
disk = ProtoPlanetaryDisk(n_disk, converter, radius_min, radius_max, q_out=25.0,
    discfraction=0.01).result
n_planets = int(planets.mass.sum() / disk[0].mass)
disk = disk.random_sample(n_disk - n_planets)
```

Since we generate the particle set `planets` independently of the gas, we randomly remove gas particles from the disk until the total mass is $0.01 M_{\odot}$. We then generate a debris disk composed of $n_{\text{debris}} = 1000$ planetesimals orbiting the star. This is a small number, but sufficient to trace the potential. The debris disk density distribution is generated in much the same way as the gas disk, except that we do not correct for the planetesimal masses, which are assumed to be negligible.

As in Section 8.6.1, we model the supernova light curve using the multiple power-law fit of Sanders et al. (2015) to supernova PS1-10a. We use parameters from their Table 3, with a peak luminosity of 2.3×10^{42} erg/s ($10^{8.8} L_{\odot}$) a few days after the explosion.

10.3.2 The Combined Solver

The first part of the calculation combines a hydrodynamics solver for the circumstellar disk, a radiative transfer code, a gravity solver for the giant planets, and a test-particle solver for the massless debris. The radiative transfer and the hydrodynamics are coupled as described in Section 8.6.1. In fact, the script for the combined hydro and radiative transfer is quite similar to the one presented in that section. It differs mainly in that the Sun is no longer a dark-matter particle in the hydro code; now it is integrated together with the giant planets using the N -body code.

³Of course, the precise Julian date is irrelevant, but ~~it was this book's 2nd edition submission deadline.~~

The debris disk is resolved using the Kepler solver in the Huayno integrator:

```
debris_gravity = Huayno(converter, number_of_workers=12)
debris_gravity.inttype = KEPLER
debris_gravity.particles.add_particles(debris)
debris_gravity.timestep = 1 | units.hour
```

Here, we start Huayno with a Kepler solver (`KEPLER = 14`) and run it on 12 cores concurrently (see Section A.3.3 for some of the other options available in Huayno). The debris disk of massless particles is subsequently sent to the N -body code, with a fixed time step of 1 hour. This is very short, but the time step in this experiment is set not by the dynamics but by the radiative transfer. Later, we could increase the time step again.

For the N -body code to integrate the planets and the Sun, we adopt `ph4`, which is initialized with

```
planet_gravity = Ph4(converter)
planet_gravity.particles.add_particles(star_and_planets)
```

After declaring the various solvers, we build the bridge, in this case composed of two gravity solvers and the radiation-hydrodynamics solver discussed in Section 8.4.2:

```
gravity_hydro = bridge.Bridge(use_threading=True)
gravity_hydro.add_system(debris_gravity, (planet_gravity, radhydro,))
gravity_hydro.add_system(planet_gravity, (radhydro,))
gravity_hydro.add_system(radhydro, (planet_gravity,))
gravity_hydro.timestep = min(1 | units.hour, time_step / 4.)
```

In the first line, we start the bridge, setting threading to `True` to speed up the calculation. In the second line, we add the `debris_gravity` code to the bridge, specifying that it is affected by `planet_gravity` and the radiative-hydrodynamics solver `radhydro`. In the third and fourth lines, we add the gravity code for the planets and the radiative-hydrodynamics solver. These are affected by each other but not by the debris disk. In the last line, we set the bridge time step to 1 hour (or even smaller), similar to the internal time step for the Kepler solver.

10.3.3 Radiative Hydrodynamics with Cooling and Heating

The simulation starts when the bright irradiation of the supernova reaches the young solar system. During this period, the heating and cooling of the disk is calculated self-consistently with the radiative transfer code. Figure 10.6 presents side and top views of the solar system 6 years after the ignition of the supernova. In the radiative-hydro calculation we assume that the gas is isothermal and free of dust. The lack of dust reduces the cooling efficiency of the disk material. With a dust mass fraction of 0.1, the equilibrium temperature in the radiation field drops by about 20%.

Once the luminosity of the supernova drops below $10^6 L_{\odot}$ (after about a year of irradiation), we switch off the radiative transfer code, because it is rather expensive to keep using it just for the cooling of the gas, and turn on the external cooling functions for the gas (see Section 6.3.4). The remaining codes continue to run as before.

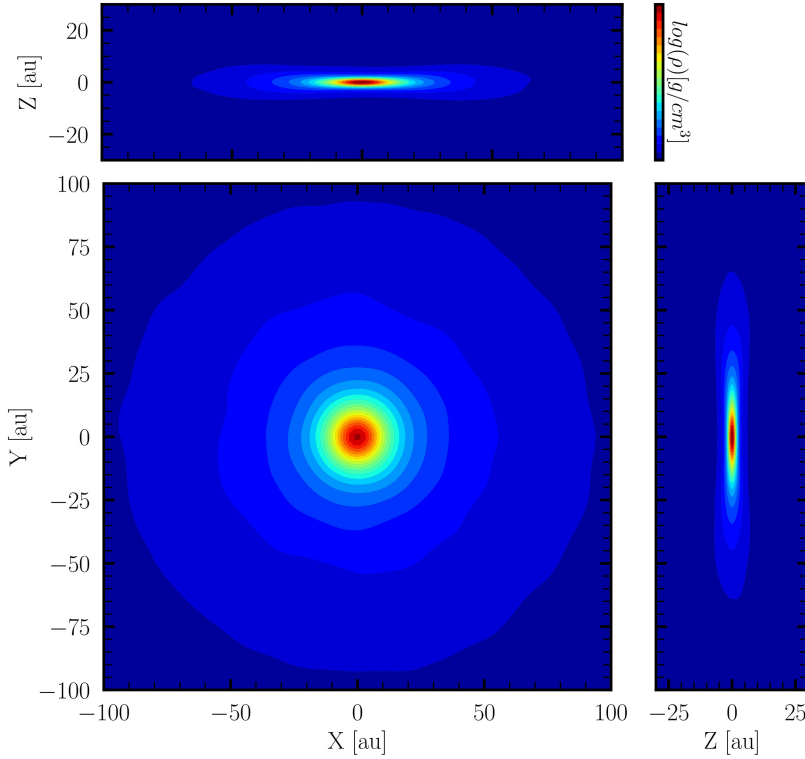


Figure 10.6. View of the solar system 6.4 years after the supernova explosion. By this time, the supernova has dimmed to such a low luminosity that there is no reason to continue the radiative transfer, but the blast wave has not yet arrived. The color bar at the top right of the figure presents a scaling of ρ ranging from $\sim 10^{-13}$ to $\sim 10^{-10} \text{ g cm}^{-3}$. The code to reproduce this result is `amuse.examples.supernova_irradiation`, which took a few days to run on a fast workstation. The figure was made using `amuse.examples.plot_projected_disk_density_from_top`.

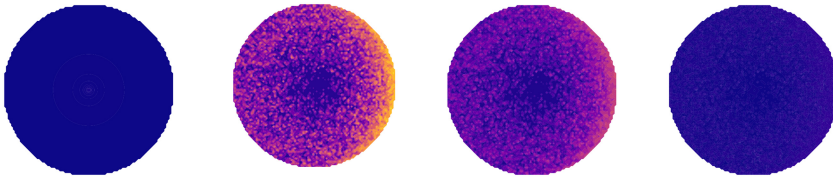


Figure 10.7. Top view of the midplane temperature of the circumstellar disk at four epochs: the start of the simulation (leftmost image); 26 days after the onset of supernova PS1-11aof; about 100 days after the supernova; and about 2 years later, when the disk has almost completely cooled down again (rightmost image). Each image was taken from the simulation where the supernova exploded at a distance of 0.2 pc and at an angle of $\theta = 6^\circ$. The figure was made using `amuse.examples.plot_projected_disk_temp_from_top`.

Figure 10.7 shows top view images of the surface temperature of the circumstellar disk at four moments in time. Although the flash of radiation does briefly raise the temperature of the disk to well over 2000 K (second image from the left in

Figure 10.7), the subsequent cooling is so efficient that by the time the supernova shock wave reaches the young solar system 10 years later, the disk has cooled back to its “normal” (presupernova) temperature of 20–200 K. In addition, the heating and cooling are so sudden that the disk hardly has time to respond, and as a result its morphology is only slightly affected by the supernova flash. The upshot is that, by the time the blast wave arrives, the disk structure is essentially the same as its initial state.

10.3.4 Injection of the Supernova Blast Wave

The hot radioactive outflow from the supernova reaches the solar system about 9.8 yr after the explosion. We inject the hot gas at a distance of 10 AU from the solar system. The hot shell is initialized in a cylinder directed at the solar system. This cylinder has a radius of $r_{\text{shell}} = 150$ AU to ensure that the entire solar system is engulfed, and the gas moves parallel to its axis. The density is uniform across the cylinder, but along the axis the density varies according to

$$\rho \propto \frac{1}{z + 10 \text{ au}}, \quad (10.1)$$

where z is distance along the axis, with $z = 0$ at the shock front. The total mass in the cylinder is

$$m_{\text{shell}} = \frac{1}{4} m_{\text{eject}} \left(\frac{r_{\text{shell}}}{d_{\text{sn}}} \right)^2. \quad (10.2)$$

For SN PS1-10a, the estimated total mass of the ejecta is $m_{\text{eject}} = 6 M_{\odot}$.

The initial temperature of the expanding supernova shell is about $T_{\text{eject}} = 9 \times 10^7$ K at a shell radius of 10 AU, but by the time the gas reaches the solar system it has cooled to (Spicer et al. 1990)

$$T_{\text{shell}} = T_{\text{eject}} \left(\frac{d_{\text{sn}}}{10 \text{ au}} \right)^{-3(\gamma-1)}, \quad (10.3)$$

so $T_{\text{shell}} \simeq 9500$ K. Here, we have adopted $\gamma = 7/5$. The sound speed in the shell is about $c_s \simeq 5.8 \text{ km s}^{-1}$, which is negligible compared to the speed of the shell at impact.

10.3.5 The Supernova Blast Wave Hits the Disk

In Figure 10.8, we present the surface density profile of the disk at three moments in time: the moment the blast wave hits the disk, 55 days later, and 3 years after the impact. The last curve (dark purple) shows the solar system’s debris disk, if we spread out all planets and planetesimals over a narrow ring. As we have just seen, the disk structure just before the blast wave hits is almost indistinguishable from the initial disk profile because the disk has cooled back to its preirradiation state. Shortly after the blast wave hits, the disk is most strongly affected near the edges by the ram pressure of the shock front, but the disk has little time to respond—the

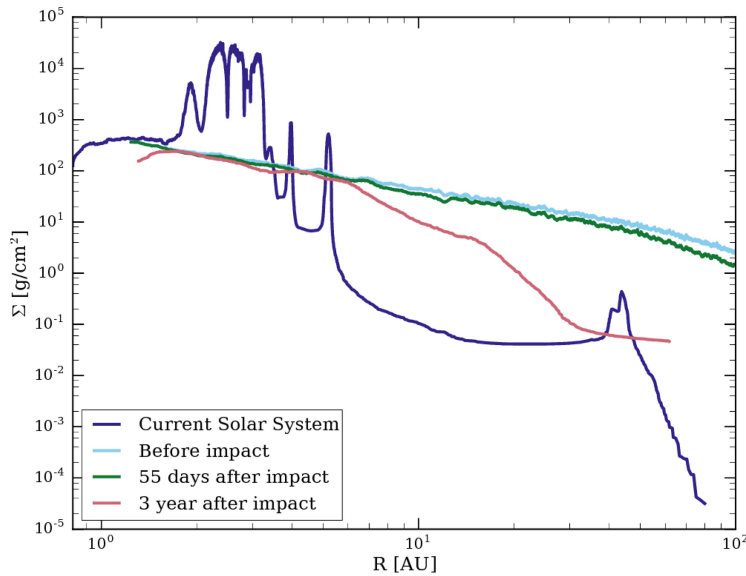


Figure 10.8. Surface density profile of the protoplanetary disk at three moments in time: just before the blast wave hits the disk (blue), 55 days later (green), and 3 years later (red). After about a year the disk has a mass of $\lesssim 0.001 M_{\odot}$. Also shown (dark purple) is the current mass distribution of planetesimals in the solar system.

shock front takes only a few weeks to cross the disk. Afterwards, the disk erodes more rapidly. Fifty-five days after the supernova blast wave hit the disk, the disk mass has dropped to half its initial value ($\sim 0.0052 M_{\odot}$, from an initial $0.01 M_{\odot}$). Most of the mass is lost from the outermost parts of the disk. After 1 year, the disk mass has dropped by an order of magnitude (to less than $0.001 M_{\odot}$).

From that point on, the disk structure remains largely unchanged, mainly due to the low density of the supernova material at late times. The final state of the disk is shown as the red curve in Figure 10.8, ~ 3 yr after first contact. We note that the planets and planetesimals are barely affected by the supernova, remaining in basically unchanged orbits around the central star. For comparison, Figure 10.8 also presents an estimate of the density profile of the current solar system's planetesimal disk. Here, we simply assume that each planetesimal contributes equally to the local density and that the solar system disk has a total mass of $0.01 M_{\odot}$, equivalent to the initially adopted disk mass. If the planetesimals had already formed by the time of the supernova shell impact, they would hardly be affected by the blast wave. It is interesting to note in Figure 10.8 that the disk becomes considerably depleted beyond 6–7 AU, where we also find a huge drop in the current planetesimal distribution. One might wonder if the current depletion beyond this range can in part be explained by such a supernova blast wave.

Figure 10.9 presents the temperature evolution of the protoplanetary disk. The temperature rises about 100 days after the supernova explosion due to the supernova flash. During this first irradiation phase, the disk becomes sufficiently hot to melt the

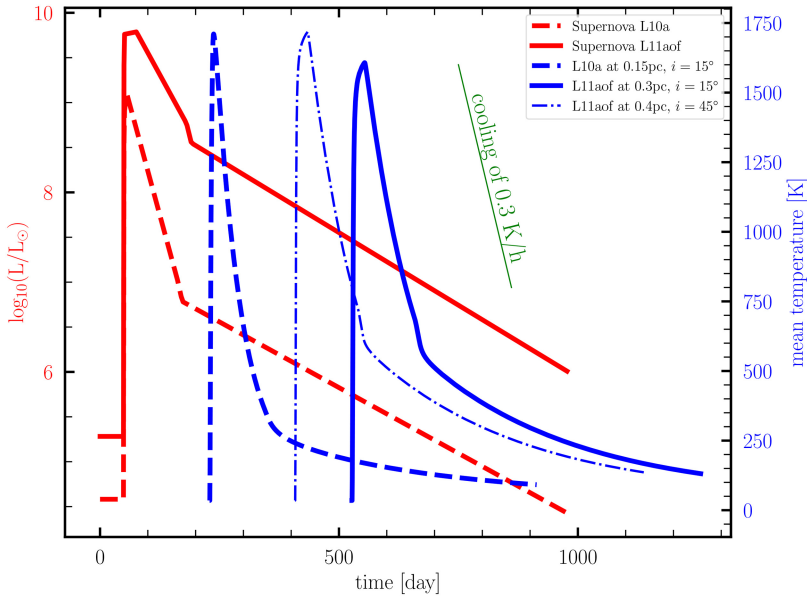


Figure 10.9. Evolution of the mean temperature of the disk due to the supernova explosion. The supernova radiation starts to heat the disk some 100 days after the explosion, and a decade later the hot nuclear blast wave starts to heat the disk. The two horizontal lines indicates temperatures of 1500 and 2000 K, which are sufficient to melt hard chondrules.

hardest chondrules. One can wonder, however, if this initial melting of the disk has any long-term consequences for the evolution of the solar system. The problem as initially posed sought to explain the enrichment by radioactive elements, which are delivered with the blast wave, but clearly the time between the irradiation and the blast wave is sufficiently long for the disk to cool down again before the shock arrives.

The nuclear blast wave hits the disk about 10 years later, heating it again to high temperatures. The blast wave does deposit some material in the Sun's protoplanetary disk, although the total amount of mass is small, less than 1% of the current disk mass. This is probably too small to explain the measured enrichment of radioactive elements in the early solar system, although these melting periods might have been important for other aspects of the formation and evolution of the solar system, such as the formation of molybdenum isotopic elements in carbonaceous and noncarbonaceous meteorites (Budde et al. 2016). A supernova explosion, therefore, could have had important consequences for the formation and evolution of the early solar system, but it is debatable whether it could have deposited a sufficient amount of short-lived radionuclides to explain the abundances of their decay products in primitive chondrules.

Finally, we note that the continuous flow of hot gas causes the gas in the surviving disk to become eccentric, and the disk starts to tilt (see also Wijnen et al. 2017) to an average inclination of about $5^\circ \pm 3$. In Figure 10.10, we show the evolution of the

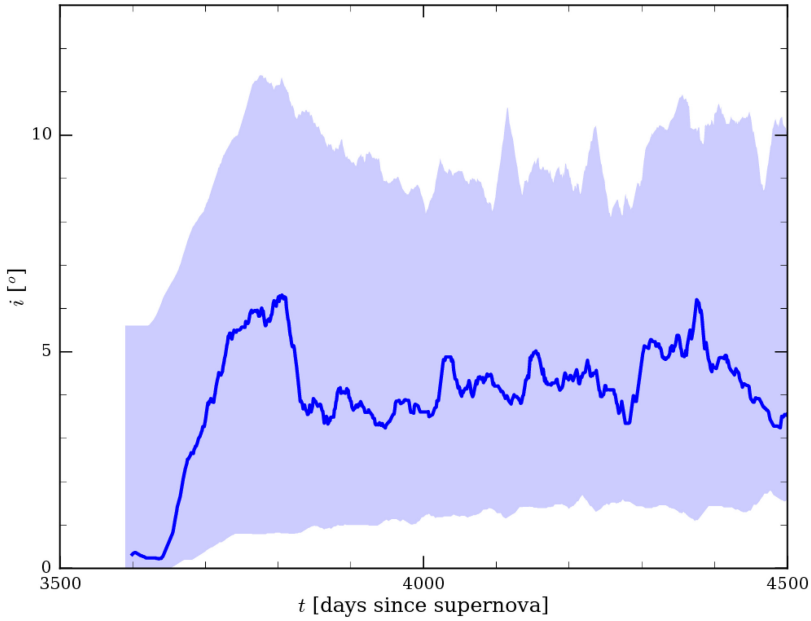


Figure 10.10. Evolution of the inclination of the planetary disk as a function of time since the disk was hit by the hot gas of the supernova blast wave. The median of the disk material inclination distribution is presented as the blue curve, with the shaded region giving the 5th and 95th percent percentiles.

inclination of the protoplanetary disk (relative to its initial value). This was measured by calculating the orbital elements for each of the gas particles and binning them in inclination using $10\sqrt{n}$ bins between zero and the maximum measured inclination. We subsequently adopted the median for display in the figure.

During the first 100 days, the median inclination of the disk grows to a maximum of about 6° before settling down to about 4° throughout the remainder of the period studied. The bulk of the disk gas has inclination between 1° and 9° . The inclination of the Sun with respect to the ecliptic plane in the solar system is $5.6^\circ \pm 1.2^\circ$, which is not inconsistent with the results from these simulations, illustrating that external events such as supernovae can plausibly account for the Sun's current inclination with respect to the ecliptic. As noted earlier, the planets were unaffected by the blast wave. If the calculation had been continued for a longer time, both the gaseous disk and the planets would probably have realigned to form a new combined disk of gas, debris, and planets. This would have led to an averaging of the angular-momentum axis. However, we did not follow this process but stopped the calculations after the impact shell traverses the disk.

The model provides a plausible explanation for the tilt of the ecliptic with respect to the Sun's equator, and also of the truncation of the outer edge of the solar system. Solving two problems with a single model is very satisfactory for researchers, particularly when complex models have trouble explaining even one of them. In this way, simulations can be used to guide the researcher's intuition, or to quantify a qualitative description of a physical process.

10.4 Accretion in the Galactic Center from S-Star Winds

The supermassive black hole in the Galactic center, Sgr A*, is orbited by a few dozen young (7–11 Myr), massive ($19\text{--}25\text{ M}_{\odot}$) stars (Schödel et al. 2002; Ghez et al. 2005), often called the “S stars.” Because their orbits are dominated by the mass of the central black hole, these stars allow an accurate measurement of the black hole’s mass (Ghez et al. 2003; Genzel et al. 2010) and give us unique insights into resonant and relativistic effects in its vicinity (see Merritt et al. 2010). The evolution of the S stars also has important implications for the local environment. With relatively high masses and advanced ages, these stars have strong stellar winds. Some of this wind material may find its way into the black hole, possibly giving rise to the X-ray emission observed by the Chandra X-ray observatory (Baganoff et al. 2003). The Chandra observations constrain the emitting region in the vicinity of Sgr A* to lie within $\sim 12,000$ AU of the central black hole (Baganoff et al. 2003). We can ask whether the winds from the S stars can account for the X-ray luminosity, or if the accreting material must come from some other population farther out.

The relatively small amount of accretion (just $\sim 10^{-7}\text{ M}_{\odot}\text{ yr}^{-1}$) onto the black hole during the pericenter passage of the nearest star, S2, led Nayakshin (2005) to argue that the S stars cannot be responsible for the general accretion onto the black hole. However, the accretion process is probably more complicated than just the wind of a single star, as was demonstrated in Cuadra et al. (2006), where the winds of the dozen S stars nearest to the black hole were resolved by hydrodynamical simulations. Recently, Lützgendorf et al. (2016) repeated this calculation with 27 S stars using the AMUSE framework. This calculation combined SPH simulations of the gas dynamics with direct N -body integration of the orbits of the S stars, including the effects of the stellar winds and accretion onto both the black hole and the S stars themselves. In this section, we show how the script used to perform these calculations was constructed.

We have encountered all of these ingredients in earlier chapters (specifically, Chapters 4 to 6). The coupling strategy was discussed in detail in Chapter 9. We repeat the Lützgendorf et al. (2016) calculation here without radiative transfer because this would put rather severe constraints on available computer resources. However, following the discussion on code-coupling strategies between hydrodynamics and radiative transfer in Chapter 7, it would be straightforward to expand the discussion to include radiative transfer if desired.

10.4.1 Initial Conditions

We start the simulation by constructing a realization of the 27 innermost S stars. The S stars are relatively massive and have most likely lost significant mass to stellar winds since their formation. Their current mass-loss rates are observed to be on the order of $10^{-5}\text{ M}_{\odot}\text{ yr}^{-1}$. In principle, knowing the masses and ages of these stars, we could simply model them using a stellar evolution code, feeding the mass loss directly into the hydro calculation. However, in practice, the masses and ages are quite poorly known. In this experiment, we adopt a simpler description that illustrates the general approach.

Figure 10.11 presents the results of a series of stellar evolution calculations using the parameterized stellar evolution code SSE (see Section 5.2.2). Reconstructing this figure should by now be relatively straightforward, given the examples in Chapter 5, and we do not provide the source code here. Note that the observed mass-loss rate for the S stars (gray bar) is consistent with winds from either 7–11 Myr old giants with initial masses between $19 M_{\odot}$ and $25 M_{\odot}$, or $18 M_{\odot}$ supergiants at an age of about 14 Myr. Accordingly, we simply assume that our model S stars were all born at the same time, each with a mass of $20 M_{\odot}$ and solar metallicity ($z = 0.02$), and are now 9 Myr old. These masses, ages, and metallicities are broadly consistent with the observed stellar parameters in the Galactic center. We evolve our model S stars to this age before integrating the equations of motion. At the start of the simulation, their masses are $18.1 M_{\odot}$, radii $890 R_{\odot}$, effective temperatures 3630 K , and luminosities $1.25 \times 10^5 L_{\odot}$. Using the theoretical wind model of Kudritzki et al. (1989)

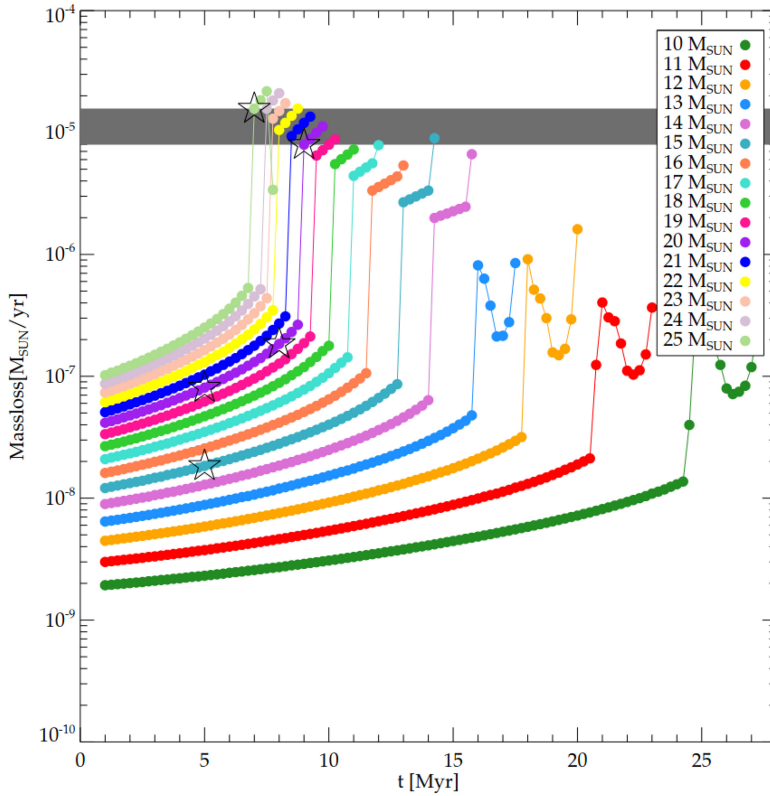


Figure 10.11. Mass-loss rates of high-mass stars, as functions of age and mass. The observed mass-loss range for the S stars, between 8×10^{-6} and $2 \times 10^{-5} M_{\odot} \text{ yr}^{-1}$ is indicated by the gray horizontal bar. To make the mass-loss rates of the stars in our simulations consistent with the observations, we opted to use $20 M_{\odot}$ stars at an age of 9 Myr (marked with an open star). Adapted with permission from Lutzgendorf et al. (2016). © 2016 The Authors. Published by Oxford University Press on behalf of the Royal Astronomical Society.

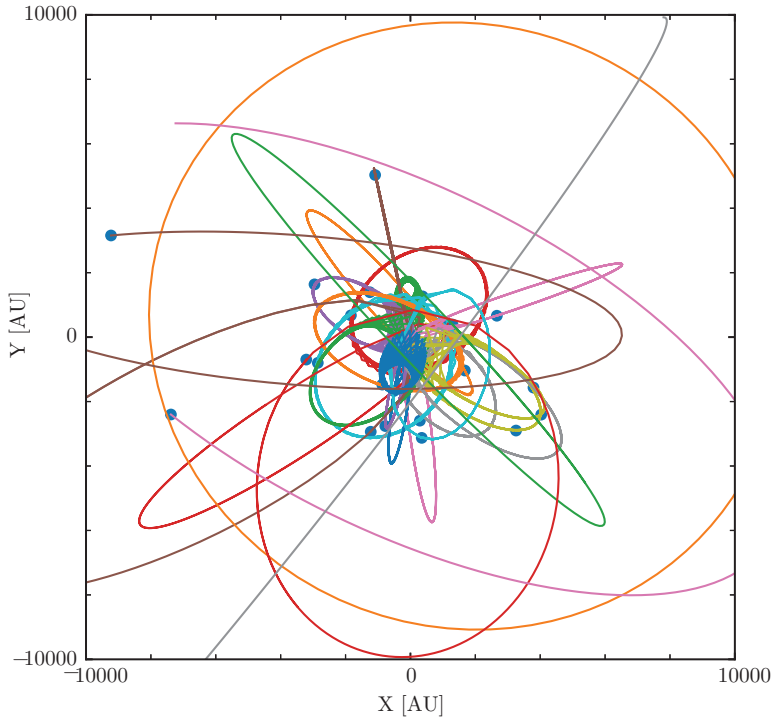


Figure 10.12. Orbital integration of the S stars over an interval of 500 yr, corresponding to the time frame of the simulation. The integration was performed using the symplectic N -body integrator Mercury (see Section 4.3). This figure was made using the script `amuse.examples.plot_sstar_orbits`.

(see Kudritzki & Puls 2000 for a review), we find a mass-loss rate of $\sim 7.9 \times 10^{-6} M_{\odot} \text{ yr}^{-1}$ with a relatively low wind velocity of about 80 km s^{-1} .

Once their physical properties are established, the stars are placed in orbit around the central supermassive black hole using the orbital parameters provided in Table 7 of Gillessen et al. (2009b), calculated using the methods discussed in Section 4.4.1. Our simulation starts with the positions and velocities of the S stars on January 1, 2001. We adopt a black hole of mass $4.45 \times 10^6 M_{\odot}$ located at the origin of the coordinate system, with zero velocity. The initial coordinates of the S stars are generated with the script `amuse.examples.initialize_sstars.py` (see also Figure 10.12).

10.4.2 The Combined Solver

We start the simulation with no interstellar gas yet present; we assume that all such gas will be created by the winds of the S stars as they orbit the central supermassive black hole. We follow the hydrodynamics of the stellar winds while integrating the orbits of the S stars in the black hole potential. Although we do not expect the stellar

parameters to change much during the integration, we run a parameterized stellar evolution code in tandem with the other codes to determine the stellar wind parameters. This makes the script more flexible, homogeneous, and self-contained.

10.4.2.1 Bridging Gravity with Hydrodynamics

For the gravitational dynamics, we use the direct N -body integrator *Mercury*. Since we are integrating only a few dozen stars, we use the single-core non-GPU enabled version of this code (see Chapter 4 for details). We adopted the initial conditions from Gillessen et al. (2009a) and converted them to Cartesian coordinates as described in Section 4.4.1. The result of integrating the initial conditions for 500 yr is presented in Figure 10.12.

The complicated boundary conditions, the winds and accretion processes, and the lack of intrinsic symmetries make a grid-based code impractical for this application. With the SPH formalism, we can relatively easily tune the stellar winds, incorporate cooling, and monitor the accretion processes, we therefore use the SPH code *Fi* to resolve the hydrodynamics in the system. Shocks (see Section 6.1.2) are an issue in these calculations, and internal temperatures can rise high enough to emit X-rays. We deal with these rising temperatures by incorporating the cooling mechanism introduced in Section 6.3.4.

The combined solver for the hydrodynamics and the gravity is constructed using *bridge* (see Chapter 9). In this case, the gravity is affected by the hydrodynamics and the hydrodynamics by the gravity, much as discussed in Code example 9.3:

```
gravity_hydro = bridge.Bridge()
gravity_hydro.add_system(gravity, (hydro,))
gravity_hydro.add_system(hydro, (gravity,))
gravity_hydro.timestep = 2.0 * hydro.parameters.timestep
```

The final line initializes the bridge time step to a conservative value.

10.4.2.2 Stellar Winds

New SPH particles are generated at runtime when stars lose sufficient mass in their stellar winds. Instead of using the simple wind description discussed in Section 6.4.3, we adopt the built-in AMUSE wind module, which is imported via

```
from amuse.ext.stellar_wind import new_stellar_wind
```

This wind module is a class, similar to but more sophisticated than the method presented in Code example 6.2 in Section 6.3.1. This built-in wind module can be started by initializing the mass of the SPH particles modeling the wind (`mass_gas`), the target particle set to which this gas should be added (here called `gas`), and the time step (`dt`). We also set an additional parameter called `derive_from_evolution`, which indicates that the winds of the stars should be derived from the running stellar evolution module,

```
wind = new_stellar_wind(
    mass_gas, target_gas=gas, timestep=time_step / 2., derive_from_evolution=True
)
```

After declaring the stellar wind class, we add the particles, in this case the particle set of $20 M_{\odot}$ stars at an age of 9 Myr (see `amuse.examples.initialize_s-stars.py`). The channel from the stars to the wind particles ensures that the wind parameters calculated in the stellar evolution code are properly copied to the particles in the wind module.

```
wind.particles.add_particles(stars)
channel_stars_to_wind = stars.new_channel_to(wind.particles)
```

In the eventual script, we update the wind module as for any other code:

```
wind.evolve_model(time)
```

10.4.2.3 The Accretion Process

During the integration of the equations of motion, we allow the black hole and the S stars to accrete from the ambient gas. As with the wind module, we use the internal sink particle solution in AMUSE (see Section 6.3.2) which can be imported by:

```
from amuse.ext.sink import new_sink_particles
```

after which we initiate the sink particles for the S stars and have them accrete:

```
sink_stars = new_sink_particles(stars)
sink_stars.accrete(gas)
```

The stellar parameters are provided by the stellar evolution code, and the stellar radius is used as the sink radius.

For the black hole, the sink is initialized with an accretion radius smaller than the pericenter distance of the innermost S star.

```
sink_BH = new_sink_particles(BH, sink_radius=accretion_radius)
sink_BH.accrete(gas)
```

Choosing the proper value for this accretion radius is nontrivial and affects the accretion rate onto the black hole. A small accretion radius is inefficient in terms of computer time, whereas a large radius causes too much mass to accrete onto the black hole. First-order estimates of the accretion radius are given by Bondi & Hoyle (1944) and Hoyle & Lyttleton (1939):

$$r_{\text{BHL}} = \frac{2GM_{\text{BH}}}{(v_{\text{wind}}^2 + c_s^2)}. \quad (10.4)$$

Here, c_s is the local sound speed, which depends on the temperature of the gas (see Equation (6.17)). Although the temperature near the black hole is very high (it is an

X-ray emitting source), the sound speed is still small compared to the speeds of the S star winds. For a stellar wind velocity of $\sim 1800 \text{ km s}^{-1}$, we estimate the accretion radius to be $\sim 2400 \text{ AU}$. For our calculation, we adopt 100 au as the radius at which accretion actually occurred—considerably smaller than the Bondi–Hoyle–Lyttleton radius, but large enough to avoid a dramatic breakdown or inefficiency of the hydrodynamics solver.

10.4.2.4 Removing Escaping Gas

Some of the gas liberated by the stars will escape the computational domain. If a gas particle drifts too far from the Galactic center, it will find that the Galactic center is isolated instead of embedded in a larger environment. Thus, if we want to resolve the hydrodynamics on the scale of the S stars, we cannot also resolve the motion of the gas on kiloparsec scales. To err on the safe side, we strip gas as soon as it passes a distance of twice the apocenter distance of the farthest S star, if its speed exceeds the local escape speed from the black hole. We calculate these two parameters once for the system:

```
system_size = (2 * stars.a * (1 + stars.ecc)).amax()
system_escape_velocity = (
    2 * constants.G * (stars.mass.sum() + BH.mass) / system_size
).sqrt()
```

For our system, these values are $\sim 0.7 \text{ parsec}$ and $\sim 760 \text{ km s}^{-1}$, respectively. Gas particles meeting these criteria are identified and removed from the simulation:

```
out_of_box = gas[gas.position.lengths() > system_size]
unbound = out_of_box[escaping.velocity.lengths() > system_escape_velocity]
escaping = unbound.select(
    lambda pos, vel: pos.dot(vel) > quantities.zero, ["position", "velocity"]
)
if escaping.is_empty():
    return
escaped_gas.add_particles(escaping)
gas.remove_particles(escaping)
```

Every time the amount of gas in the simulation changes (increases due to stellar winds, decreases by accretion or escape), the hydrodynamics solver is updated. We realize this by synchronizing the local gas particles with the particles in the hydrodynamics solver:

```
gas.synchronize_to(hydro.gas_particles)
```

10.4.2.5 The Event Loop

After declaring and initializing all the separate ingredients, we can write the event loop for this simulation. It is not complicated, but rather long due to the accumulation of the separate pieces. We have removed the diagnostics, validation, and data-writing operations for brevity and to clarify the basic functionality. But of course, these checks are of the utmost importance for a production simulation.

```

while time < time_end:
    time = time + time_step / 2
    stellar.evolve_model(pretime+time)
    time = time + time_step / 2
    gravity_hydro.evolve_model(time)
    stellar.evolve_model(pretime+time)

    channel_stars_to_wind.copy()
    wind.evolve_model(time)
    gas.synchronize_to(hydro.gas_particles)

    sink_BH.accrete(gas)
    gas.synchronize_to(hydro.gas_particles)
    sink_stars = new_sink_particles(stars)

    sink_stars.accrete(gas)
    gas.synchronize_to(hydro.gas_particles)
    remove_escaping_gas(gas, system_size, system_escape_velocity,
                        escaped_gas)
    gas.synchronize_to(hydro.gas_particles)

```

The sinks keep track of the amount of angular momentum accreted by the black hole particle in order to estimate the size of the accretion disk.

```

m_accreted = BH_sink.particles.mass.sum()
diskrad = (
    sink_BH.angular_momentum.length()**2
    / (constants.G * m_accreted**2 * BH.mass[0])
)

```

Running this script for any length of time and at a reasonable resolution of at least $5 \times 10^{-8} M_{\odot}$ per gas particle requires a substantial amount of computer time, really requiring the use of a large computing cluster or supercomputer rather than a laptop or workstation. Figure 10.13 shows the simulation results as published in Lützgendorf et al. (2016).

10.4.3 Results of the Simulation

The highest resolution run performed by Lützgendorf et al. (2016) was for a gas-particle mass of $4.8 \times 10^{-8} M_{\odot}$, with new SPH particles generated during each time step of 1 yr. It lasted for 500 yr, which is about 3 times the orbital period of the star S8. Figure 10.13 presents several snapshots of this simulation. As discussed previously, the initial snapshot (upper left) contains no gas; it takes a few hundred years for the system to reach a richer gas distribution. The simulation

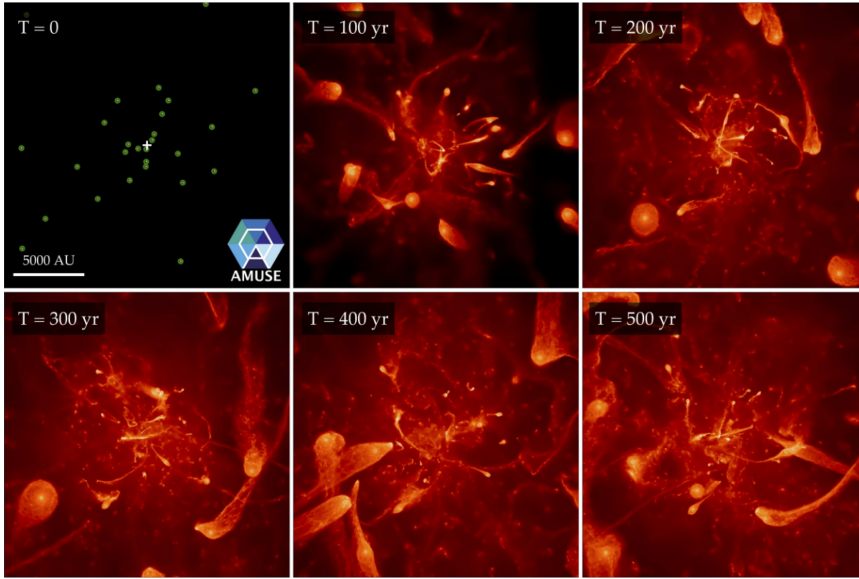


Figure 10.13. Evolution of the S stars in the Galactic center. This is Figure 3 from Lützgendorf et al. (2016); a spectacular animation of this simulation can be found at <https://youtu.be/soGFgzRso3c>. Adapted with permission from Lützgendorf et al. (2016). © 2016 The Authors. Published by Oxford University Press on behalf of the Royal Astronomical Society.

was stopped at 500 yr because, although the gas distribution continues to change, the accretion onto the central black hole stabilizes at $\sim 1.4 \times 10^{-7} M_{\odot} \text{ yr}^{-1}$. This slightly exceeds the observed accretion rate of 10^{-8} – $10^{-7} M_{\odot} \text{ yr}^{-1}$ (Baganoff et al. 2003; Xu et al. 2006).

Figure 10.14 presents the total amount of mass accreted by the black hole during several simulations. The various curves show the result of different calculations in which we varied the ages of the S stars. As discussed earlier, the ages are important for determining the S star winds, and therefore accretion onto the black hole. If the S stars are young ($\lesssim 8$ Gyr), they are unable to account for the observed accretion onto the black hole, but if they are giants with ages $\gtrsim 9$ Myr, the observed accretion rate onto the black hole could be explained by the S star winds.

In every simulation, the contribution of star S2 provides at most $\sim 2\%$ of the total accretion rate onto the black hole. The idea that S2, being the nearest star, contributes most to the accretion onto the black hole is clearly not valid. During our 500 yr simulation, the star S14 contributed most mass to the black hole—almost an order of magnitude more than S2—even though S2 has the smallest pericenter distance. Of course, our assumption that all of the S stars have the same mass and age is artificial and somewhat contrived. When improved observations of the S stars provide better constraints on their masses and evolutionary states, we can easily improve our treatment of accretion onto the black hole.

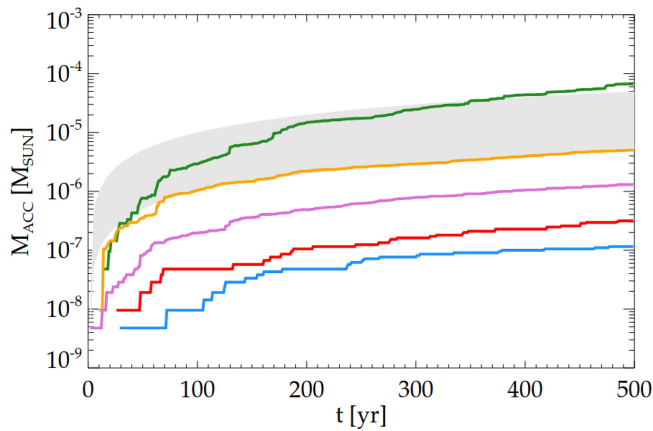


Figure 10.14. Total amount of mass accreted by the central supermassive black hole, as a function of time. The gray shaded area is an estimate of the total amount of mass accreted by the black hole, using observed mass accretion rates (Baganoff et al. 2003; Xu et al. 2006). The curves represent calculations for 20 $M_{\odot}$ stars at ages of 5 Myr (red curve), 8 Myr, (purple curve) and 9 Myr (green curve). The other two curves are for 15 $M_{\odot}$ stars at 5 Myr (blue) and 25 $M_{\odot}$ at 7 Myr (yellow). Adapted with permission from Lutzgendorf et al. (2016). © 2016 The Authors. Published by Oxford University Press on behalf of the Royal Astronomical Society.

10.5 Visualization

10.5.1 Fresco: Visualizing Gas-Rich Dusty Star Clusters

Fresco is a visualization package specifically for making “observational” images of simulations, focused on simulations in which stellar evolution, stellar dynamics, and possibly gas dynamics are incorporated. It takes the luminosities and temperatures from stars, and uses these to calculate their brightness in a series of color bands (by default the Bessell *UBVRI* bands). Then, using the positions of the stars, it calculates a base image for each color band, which is then convolved with a point-spread function (PSF). Finally, the color bands are combined to create an RGB image, by default mimicking the colors as the human eye would see them.

An additional feature of **Fresco** is the ability to include extinction from interstellar dust, using gas particles as a proxy. In this case, it creates a three-dimensional extinction map using the **FiMap** module (part of **Fi**), by default using a fixed gas-to-dust ratio, though this can be customized. After the brightness of each star is calculated in each color band, the extinction is then applied before convolving with the PSF. Additionally, the dust can reflect a fraction of the nearby starlight, using a tweakable albedo parameter.

Fresco is not in the main **amuse** GitHub repository, but can be downloaded from <https://github.com/amusecode/amuse-fresco>. An example of an image made using **Fresco** is presented in Figure 10.15, which was generated with exactly the same data as were used to generate Figure 6.10.

spo

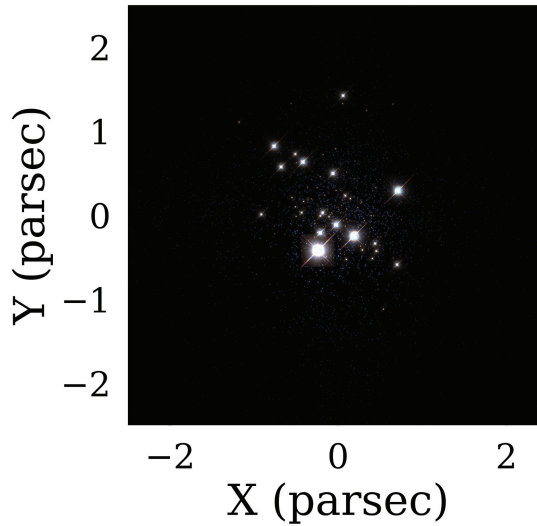


Figure 10.15. Projection of a collapsing molecular cloud at an age of 4.5 Myr. The stars are represented with the PSF of the Hubble Space Telescope, and the underlying gas distribution is given in contours. The data from which this figure is made is identical to those used to generate Figure 6.10.

10.6 Closing

You have reached the end of the final chapter of this book. Congratulations! We have discussed several examples on the use of AMUSE, and we gave a number of pointers to other projects in which the framework is used. Several of these examples are published in peer-reviewed journals. They generally address somewhat more complex problems, but the fundamental working remains unchanged. What is left now, are the appendices and some philosophical concluding remarks in Part III.

References

- Appel, S. M., Andersson, E. P., Burkhardt, B., et al. 2025, AAS Meeting, 245, 343.08
 Baganoff, F. K., Maeda, Y., Morris, M., et al. 2003, *ApJ*, 591, 891
 Bondi, H., & Hoyle, F. 1944, *MNRAS*, 104, 273
 Borgdorff, J., Lorenz, E., Hoekstra, A. G., Falcone, J. L., & Chopard, B. 2011, 7th IEEE International Conference on e-Science (Los Alamitos, CA: IEEE Computer Society Press) 97
 Budde, G., Burkhardt, C., Brenneka, G. A., et al. 2016, *E&PSL*, 454, 293
 Chambers, J. E., & Migliorini, F. 1997, AAS/Division for Planetary Sciences Meeting Abstracts, 29, 27.06
 Chevalier, R. A. 2000, *ApJL*, 538, L151
 Cuadra, J., Nayakshin, S., Springel, V., & Di Matteo, T. 2006, *MNRAS*, 366, 358
 Dobbs, C. L., & Pringle, J. E. 2013, *MNRAS*, 432, 653
 Draine, B. T. 2011, *ApJ*, 732, 100
 Fryxell, B., Olson, K., Ricker, P., et al. 2000, *ApJS*, 131, 273
 Genzel, R., Eisenhauer, F., & Gillessen, S. 2010, *RvMP*, 82, 3121

- Ghez, A. M., Duchêne, G., Matthews, K., et al. 2003, [ApJL](#), **586**, L127
- Ghez, A. M., Salim, S., Hornstein, S. D., et al. 2005, [ApJ](#), **620**, 744
- Gillessen, S., Eisenhauer, F., Trippe, S., et al. 2009a, [ApJ](#), **692**, 1075
- Gillessen, S., Eisenhauer, F., Fritz, T. K., et al. 2009b, [ApJL](#), **707**, L114
- Hoyle, F., & Lyttleton, R. A. 1939, [PCPS](#), **35**, 405
- Kepler, J. 1609, *Astronomia Nova* (Heidelberg: G. Voegelinus), 1
- Kudritzki, R. P., Pauldrach, A., Puls, J., & Abbott, D. C. 1989, [A&A](#), **219**, 205
- Kudritzki, R.-P., & Puls, J. 2000, [ARA&A](#), **38**, 613
- Lewis, S., McMillan, S., Mac Low, M. M., et al. 2023, AAS Meeting, 241, 09.03D
- Liow, K. Y., Rieder, S., Dobbs, C. L., & Jaffa, S. E. 2022, [MNRAS](#), **510**, 2657
- Looney, L. W., Tobin, J. J., & Fields, B. D. 2006, [ApJ](#), **652**, 1755
- Lützgendorf, N., Helm, E., van der, Pelupessy, F. I., & Portegies Zwart, S. 2016, [MNRAS](#), **456**, 3645
- Luu, T.-H., Young, E. D., Gounelle, M., & Chaussidon, M. 2015, [PNAS](#), **112**, 1298
- Merritt, D., Alexander, T., Mikkola, S., & Will, C. M. 2010, [PhRvD](#), **81**, 062002
- Nayakshin, S. 2005, [A&A](#), **429**, L33
- Ouellette, N., Desch, S. J., & Hester, J. J. 2007, [ApJ](#), **662**, 1268
- Pearson, S. G., & McCaughrean, M. J. 2023, [arXiv:2310.01231](#)
- Pelupessy, F. I., Jänes, J., & Portegies Zwart, S. 2012, [NewA](#), **17**, 711
- Portegies Zwart, S. F., & Verbunt, F. 1996, [A&A](#), **309**, 179
- Portegies Zwart, S., Pelupessy, I., Martínez-Barbosa, C., van Elteren, A., & McMillan, S. 2020, [CNSNS](#), **85**, 105240
- Price, D. J., Wurster, J., Tricco, T. S., et al. 2018, [PASA](#), **35**, e031
- Rein, H., & Liu, S. F. 2012, [A&A](#), **537**, A128
- Rieder, S. 2019, rieder/ekster: IAUS 351, v1.0.1a, Zenodo, [doi:10.5281/zenodo.3356893](#)
- Rieder, S., & Liow, K. Y. 2021, Zenodo, [doi:10.5281/zenodo.5520944](#), v1.01
- Rieder, S., Dobbs, C., Bending, T., Liow, K. Y., & Wurster, J. 2022, [MNRAS](#), **509**, 6155
- Sanders, N. E., Soderberg, A. M., Gezari, S., et al. 2015, [ApJ](#), **799**, 208
- Schiller, M., Paton, C., & Bizzarro, M. 2015, [GeCoA](#), **149**, 88
- Schödel, R., Ott, T., Genzel, R., et al. 2002, [Natur](#), **419**, 694
- Scott, E. R. D. 2007, [AREPS](#), **35**, 577
- Sills, A., Rieder, S., Buckner, A. S. M., et al. 2023, [MNRAS](#), **519**, 4142
- Spicer, D. S., Clark, R. W., & Maran, S. P. 1990, [ApJ](#), **356**, 549
- Stahler, S. W., & Palla, F. 2005, *The Formation of Stars* (New York: Wiley)
- Teyssier, R. 2002, [A&A](#), **385**, 337
- van Elteren, A., Portegies Zwart, S., Pelupessy, I., Cai, M. X., & McMillan, S. L. W. 2019, [A&A](#), **624**, A120
- Wall, J. E., Mac Low, M.-M., McMillan, S. L. W., et al. 2020, [ApJ](#), **904**, 192
- Wang, L., Iwasawa, M., Nitadori, K., & Makino, J. 2020, [MNRAS](#), **497**, 536
- Wijnen, T. P. G., Pelupessy, F. I., Pols, O. R., & Portegies Zwart, S. 2017, [A&A](#), **604**, A88
- Wilhelm, M. J. C., & Portegies Zwart, S. 2024, [A&A](#), **691**, A71
- Xu, Y.-D., Narayan, R., Quataert, E., Yuan, F., & Baganoff, F. K. 2006, [ApJ](#), **640**, 319